

PROJECT DOCUMENTATION

Regal Extract

ICEGATE Shipping-Bill PDF → Living Excel Master

Project Name	Regal Extract
Version	1.0
Developed For	3S Pharmaceuticals India Pvt. Ltd.
Document Date	June 2026
Classification	Internal / Confidential

1. Executive Summary

Regal Extract is a private, locally-hosted web application designed to automate the extraction of shipping-bill data from ICEGATE customs PDFs into a structured, editable Excel master workbook. The application eliminates manual data entry for export documentation workflows at 3S Pharmaceuticals India Pvt. Ltd.

Project Objective

The solution is built to:

- Automatically extract key fields from ICEGATE Shipping Bill PDFs using fixed-layout text rules
- Preserve multi-currency amounts (USD, EUR, GBP, JPY) with correct Excel number formats
- Compute INR value as a live Excel formula ($\text{₹Foreign Amount} \times \text{Exchange Rate}$), not a static number
- Append or update rows in an existing Excel master without disturbing user-created columns or formulas
- Operate entirely offline — no cloud upload, no paid API, no subscription required

Key Design Principles

Principle	Description
Rule-based extraction	Fixed coordinate and regex patterns; no AI model or internet dependency
Privacy-first	All processing happens on the user's own machine; nothing leaves the LAN
Non-destructive append	Only recognized headers are written; existing user columns and formulas are preserved
Duplicate-safe	Existing Bill No. rows are updated in-place; new records are appended above the totals row
Atomic saves	The master workbook is written to a temp file and renamed atomically; data is never left in a half-written state

2. What is Regal Extract?

Regal Extract is a three-step browser-based workflow that runs on the user's local machine. The user opens a simple web page, uploads ICEGATE Shipping Bill PDFs, reviews the extracted data in an editable table, and appends the reviewed rows to an Excel master workbook — all without leaving the browser and without any data touching the internet.

System Architecture Flow

The application follows a clean single-server architecture:

Browser (UI) → Local HTTP Server (Python) → PDF Extractor → Workbook Service → Excel Master

Component	File	Responsibility
HTTP Server	app.py	Serves the web UI; handles API endpoints for /api/master, /api/extract, /api/append, /api/download/master
PDF Extractor	extractor.py	Opens PDFs with pdfplumber; applies coordinate windows and regex patterns to pull field values
Workbook Service	workbook_service.py	Reads the Excel master, locates headers by name, appends or updates rows, adjusts the Excel Table range, and saves atomically
Web Front-end	static/	Single-page app (HTML + JS + CSS); communicates with the server via fetch() API calls

3. Business Problem Statement

Existing Situation

Exporters filing shipping bills through ICEGATE receive multi-page PDF documents that contain customs-clearance data: Shipping Bill number and date, port code, invoice details, exchange rates, FOB value, drawback amounts, IGST, and RoDTEP figures. The finance team at 3S Pharmaceuticals currently extracts these values manually and types them into a master Excel register.

Current Challenges

- **High volume:** Manual Effort

Each shipment requires reading a 2-4 page ICEGATE PDF and transferring 12–15 values by hand, consuming significant staff time.

- **Data entry risk:** Human Errors

Invoice numbers, exchange rates, and duty amounts are prone to transcription mistakes that can affect drawback claims and accounting reconciliations.

- **Multi-currency invoices:** Currency confusion

Invoices are raised in USD, EUR, GBP, or JPY. Manually recording the currency symbol alongside the amount is error-prone; the correct Excel number format is often not applied.

- **INR computation:** Formula breakage

The INR column must stay a live formula (Foreign Amount × Exchange Rate) so that exchange rate corrections ripple through automatically. Manual paste-as-value breaks this.

- **Accidental overwrite:** Master integrity risk

Manually pasting into the master workbook risks overwriting user-created analysis columns, formula rows, or the totals row.

Business Impact

- Increased risk of incorrect drawback or RoDTEP claims due to keying errors
- Delayed month-end closing while the register is being populated manually
- Inconsistent column formatting making pivot analysis unreliable
- Inability to scale as export volume grows without adding headcount

4. Proposed Solution

Regal Extract replaces the manual entry workflow with a three-step guided process accessible from any browser on the same computer running the application.

Step 1 — Configure the Master Workbook

- Upload an existing master Excel file, or create a fresh pre-formatted workbook with the 14 standard export columns.
- The application scans the workbook for recognized column headers (by name, not column letter) and confirms readiness.
- Any previously stored master is automatically backed up to data/backups/ with a timestamp before it is replaced.

Step 2 — Upload and Extract PDFs

- The user selects one or more ICEGATE Shipping Bill PDFs via drag-and-drop or file picker.
- Each PDF is processed server-side: page text is extracted with pdfplumber, then coordinate-bounded windows and regular expressions identify the 15 target fields.
- An extraction confidence score (0–100 %) is computed from the fraction of required fields successfully read; any unread field is listed as a warning.
- Results are displayed in an editable review table — the user can correct any field before committing.

Step 3 — Review and Append

- After reviewing, the user clicks “Append to master Excel”.
- If a Bill No. already exists in the master, that row is updated in-place (duplicate-safe mode).
- New invoice numbers are inserted above the totals row; an Excel Table range is extended automatically.
- The INR column is written as a live formula; the foreign amount cell receives the correct currency number format.
- The updated workbook is saved atomically and is immediately available for download.
- An audit log (data/audit.jsonl) records every append with a timestamp and a list of rows changed.

5. Technology Stack

Runtime Requirements

Component	Version	Purpose
Python	3.11 or newer	Application runtime; standard library HTTP server
pdfplumber	≥ 0.11, < 1	PDF text extraction with word-level coordinate data
openpyxl	≥ 3.1, < 4	Excel workbook read/write; formula translation; table range management
pypdf	≥ 5, < 7	Supplementary PDF utilities (listed in requirements.txt)
Browser	Any modern browser	User interface; no framework or install needed

No External Dependencies

Regal Extract deliberately avoids external services:

- No AI API — all extraction is rule-based (regex + coordinate windows)
- No cloud storage — files remain on the user's machine in the data/ directory
- No email server or notification service
- No database — the Excel master workbook is the single source of truth
- No NPM build step for the front-end — plain HTML, CSS, and vanilla JavaScript

Data Files and Storage

Path	Contents
data/master.xlsx	The active master workbook (replaced atomically on each append)
data/backups/	Timestamped copies of the master created before each replacement or new-master action
data/pending/	Temporary batch JSON files written between extract and append steps
data/audit.jsonl	Append-only JSON-lines audit log; one entry per append operation with timestamp and row-level details

6. Extracted Fields and Excel Mapping

The extractor reads the following 15 fields from each ICEGATE PDF. All 14 data fields map directly to a column in the master workbook; the source filename is retained internally for audit purposes.

Field Key	Master Column Header	Excel Format	Extraction Method
particulars	Particulars	Text	Consignee name from page-1 header or coordinate window on page 1/2
bill_no	Bill No.	Number (integer)	Invoice number from page-2 regex or summary region on page 1
invoice_date	Date	d-mmm-yy	Date parsed from page-2 row; multiple format patterns supported
sb_no	SB NO.	Text	Shipping Bill number from ICES header line on page 1
sb_date	SB DATE	d-mmm-yy	SB date from ICES header line on page 1
port_code	Port Code	Text (upper)	Port code from ICES header line on page 1
foreign_amount	\$ / E	\$#,##0.00 or €/ £/ ¥	Amount from invoice details section; currency symbol set from currency code
exchange_rate	Exch Rate	0.0000	Exchange rate from coordinate window on page 2 or INR-rate regex
inr	INR	₹#,##0.00	Written as Excel formula =Foreign_Amount × Exchange_Rate (NOT a static value)
taxable_value	Value@5%	₹#,##0.00	IGST taxable value from page-1 coordinate window or fallback regex
igst	IGST@5%	₹#,##0.00	IGST amount from page-1 coordinate window
fob	FOB	₹#,##0.00	FOB value from page-1 coordinate window
drawback	DBK	₹#,##0	Drawback claim from page-1 coordinate window or fallback regex
rodtep	RoDTEP	₹#,##0	RoDTEP amount from page-1 coordinate window or fallback regex

Note: The INR row is highlighted because it is a computed Excel formula, not an extracted value. Changing the exchange rate in the Exch Rate column will automatically recalculate INR.

7. Workbook Safety Rules

These rules govern how Regal Extract reads and writes the Excel master workbook to ensure data integrity:

- **Header-name lookup:** Column location by header name, not letter

The workbook service scans all rows up to row 60 across all sheets to find the row that has the highest count of recognized headers. This means the master can be reformatted (columns reordered, new columns inserted) without breaking the app.

- **Flexible matching:** Recognized headers and aliases

Each of the 14 fields has a set of normalized aliases (e.g., “Drawback”, “DBK”, and “Drawback Amount” all resolve to the drawback field). Case and punctuation are ignored during matching.

- **Column protection:** Existing user columns are never cleared

Only columns whose header is in the recognized alias list are written. Any additional columns the user has created (analysis columns, notes, flags) are ignored entirely.

- **Formula carry-forward:** Formulas in user columns flow into new rows

When a genuinely new row is inserted, the workbook service copies the cell style and, for formula cells in user-created columns, translates the formula reference to the new row number using openpyxl’s Translator.

- **Totals row safety:** Totals row detection

The service scans for a row containing the word “total” (case-insensitive) in any cell. New rows are always inserted above this row, and formulas in the totals row that reference a range are extended to include the new row.

- **Data integrity:** Atomic file save

The workbook is written to a temporary file in the same directory as master.xlsx, then `os.replace()` performs an atomic rename. The master is never left in a partially-written state.

- **Rollback safety:** Backup on replacement

Whenever a new master is uploaded or a new master is created while one already exists, the current master.xlsx is copied to data/backups/ with a datetime suffix before any change is made.

8. How to Run the Application

First-Time Setup (Windows)

1. Install Python 3.11 or newer from python.org (ensure “Add Python to PATH” is checked).
2. Extract the Regal Extract folder to any location on your computer.
3. Double-click setup_and_start.bat.
 - This installs the three open-source libraries (pdfplumber, openpyxl, pypdf) from PyPI.
 - After a successful install, the app starts and opens <http://127.0.0.1:8765> in your default browser.

Subsequent Starts

Double-click start_app.bat. The app starts without re-installing packages.

The server prints a log line for every HTTP request. Close the command window or press Ctrl+C to stop the server.

Environment Variables (Optional)

Variable	Default	Effect
REGAL_HOST	127.0.0.1	Bind address; set to 0.0.0.0 to allow LAN access
REGAL_PORT	8765	TCP port the server listens on
REGAL_NO_BROWSER	(not set)	Set to 1 to suppress automatic browser launch

9. API Endpoint Reference

The local HTTP server exposes the following endpoints, all accessed by the browser front-end:

Method + Path	Auth	Description
GET /	None	Serves the main index.html web application page
GET /api/status	None	Returns JSON with master workbook readiness, record count, and recognized column count
GET /api/download/master	None	Streams the current master.xlsx as a file download
POST /api/master	None	Multipart form; action=new creates a blank formatted master; action=existing installs an uploaded Excel file as the master
POST /api/extract	None	Multipart form; accepts one or more PDFs; returns extracted field JSON plus a batch_id for the subsequent append call
POST /api/append	None	JSON body; receives reviewed rows from the browser; appends/updates the master; returns added/updated counts and download link

All endpoints return JSON with an ok boolean. On error, a descriptive error string is included and HTTP 400 is returned.

10. Limitations and Known Constraints

Area	Limitation
PDF layout dependence	Extraction rules are calibrated for the standard ICEGATE Shipping Bill layout. Non-standard or heavily reformatted PDFs may yield lower confidence scores or missing fields.
No OCR support	The app uses text-layer extraction. Scanned PDFs (image-only) will return blank fields. A confidence score of 0 % indicates this condition.
Single-user / local only	The server binds to localhost by default. Multi-user or network access requires setting REGAL_HOST=0.0.0.0, and the Excel file cannot be shared concurrently.
Windows-only launcher	The .bat files are for Windows. On macOS or Linux, run python app.py directly from a terminal after installing requirements.
Upload size limit	A single upload request is capped at 80 MB. For large batches, split PDFs into multiple upload sessions.
No user authentication	The app has no login page. Access is controlled by network binding only (localhost by default).

11. Conclusion

Regal Extract solves a specific, high-frequency manual task at 3S Pharmaceuticals: transcribing ICEGATE shipping-bill data into an Excel export register. By applying fixed-layout extraction rules, the application achieves reliable field capture without any cloud dependency, AI subscription, or complex infrastructure.

The three-step workflow (configure master → upload PDFs → review and append) keeps the user in control at every stage. The review table allows manual correction before any data touches the workbook, and the non-destructive append logic ensures that the user's existing formatting, formulas, and analysis columns are never disturbed.

The application is intentionally minimal: a single Python process, three open-source libraries, and a plain HTML front-end. This makes it easy to understand, audit, and maintain without specialized DevOps knowledge.

Expected Outcomes

- Near-elimination of manual data entry for shipping-bill register updates
- Consistent Excel number formats for currency and date columns across all records
- Live INR formula that recalculates automatically when exchange rates are corrected
- Full audit trail of every append operation in data/audit.jsonl
- Zero risk of accidental overwrite of user-created columns or totals rows
- Fast adoption curve due to simple browser-based interface and one-click start

12. Project File Structure

File / Folder	Purpose
app.py	Main entry point; HTTP server handling all routes and request parsing
extractor.py	PDF field extraction logic; coordinate windows, regex patterns, confidence scoring
workbook_service.py	Excel master read/write; header discovery; append/update logic; atomic save
requirements.txt	Python package dependencies (pdfplumber, openpyxl, pypdf)
setup_and_start.bat	Windows first-run script: installs packages then starts the server
start_app.bat	Windows subsequent-run script: starts the server without reinstalling
static/index.html	Web application HTML; three-section workflow UI
static/app.js	Front-end JavaScript; fetch() API calls; review table rendering; file handling
static/styles.css	Application styles; teal/gold design language; responsive layout
data/master.xlsx	Active master workbook (auto-created at runtime; not in the ZIP)
data/backups/	Timestamped master backups (auto-created at runtime)
data/audit.jsonl	Append-only audit log (auto-created at runtime)